Rod Ciombor
5/10/26
CIS-2532-NET01
Dr. Sheikh Shamsuddin
Week 13 Lab

Import Libraries

In [6]:
```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report
```

Prepare Data

In [7]:
```
# Create synthetic vehicle data
np.random.seed(42)

# Generate features: weight (kg), engine_size (L), num_doors
n_samples = 1000

# Cars
car_weight = np.random.normal(1500, 200, n_samples // 2)
car_engine = np.random.normal(2.0, 0.5, n_samples // 2)
car_doors = np.random.choice([2, 4], n_samples // 2)

# Trucks
truck_weight = np.random.normal(3000, 400, n_samples // 2)
truck_engine = np.random.normal(5.0, 1.0, n_samples // 2)
truck_doors = np.random.choice([2, 4], n_samples // 2)

# Combine data
X = np.vstack([
    np.column_stack([car_weight, car_engine, car_doors]),
    np.column_stack([truck_weight, truck_engine, truck_doors])
])

# Create labels: 0 for car, 1 for truck
y = np.array([0] * (n_samples // 2) + [1] * (n_samples // 2))
```

Split data into Training and Testing Sets

In [8]:
```
# Split data: 80% training, 20% testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(f"Training samples: {X_train.shape[0]}")
print(f"Testing samples: {X_test.shape[0]}")
```

```
Training samples: 800
Testing samples: 200
```

Scale Features

In [9]:
```
#Ensure all features contribute equally to the model
#Only do this on training data

# Initialize the scaler
scaler = StandardScaler()

# Fit on training data and transform both sets
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Train Classifier

In [10]:
```
# Create a Support Vector Machine classifier
classifier = SVC(kernel='rbf', random_state=42)

# Train the model
classifier.fit(X_train_scaled, y_train)

print("Model training complete!")
```

```
Model training complete!
```

Make Predictions and Evaluate

In [11]:
```
# Make predictions on test data
y_pred = classifier.predict(X_test_scaled)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy:.2%}")

# Detailed classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred,
                            target_names=['Car', 'Truck']))
```

```
Model Accuracy: 99.50%

Classification Report:
              precision    recall  f1-score   support

         Car       1.00      0.99      0.99        96
       Truck       0.99      1.00      1.00       104

    accuracy                           0.99       200
   macro avg       1.00      0.99      0.99       200
weighted avg       1.00      0.99      0.99       200
```

Use Classifiers for New Predictions

In [12]:
```
# New vehicle data
new_vehicle = np.array([[1800, 2.5, 4]])  # weight, engine_size, doors

# Scale the new data
new_vehicle_scaled = scaler.transform(new_vehicle)

# Make prediction
prediction = classifier.predict(new_vehicle_scaled)
probability = classifier.decision_function(new_vehicle_scaled)

vehicle_type = "Car" if prediction[0] == 0 else "Truck"
print(f"Predicted vehicle type: {vehicle_type}")
print(f"Confidence score: {abs(probability[0]):.2f}")
```

```
Predicted vehicle type: Car
Confidence score: 1.16
```

Cross Validation

In [13]:
```
from sklearn.model_selection import cross_val_score

# Perform 5-fold cross-validation
scores = cross_val_score(classifier, X_train_scaled, y_train, cv=5)
print(f"Cross-validation scores: {scores}")
print(f"Average CV accuracy: {scores.mean():.2%} (+/- {scores.std() * 2:.2%})")
```

```
Cross-validation scores: [1.      1.      0.99375 1.      1.     ]
Average CV accuracy: 99.88% (+/- 0.50%)
```

Hyperparameter Tuning

In [14]:
```
from sklearn.model_selection import GridSearchCV

# Define parameter grid
param_grid = {
    'C': [0.1, 1, 10, 100],
    'gamma': ['scale', 'auto', 0.001, 0.01],
    'kernel': ['rbf', 'poly', 'sigmoid']
}

# Grid search with cross-validation
grid_search = GridSearchCV(SVC(), param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train_scaled, y_train)

print(f"Best parameters: {grid_search.best_params_}")
print(f"Best accuracy: {grid_search.best_score_:.2%}")
```

```
Best parameters: {'C': 0.1, 'gamma': 'scale', 'kernel': 'rbf'}
Best accuracy: 99.88%
```

Feature Performance Analysis

In [15]:
```
from sklearn.ensemble import RandomForestClassifier

# Train a Random Forest for feature importance
rf_classifier = RandomForestClassifier(n_estimators=100, random_state=42)
```

```
rf_classifier.fit(X_train_scaled, y_train)

# Get feature importance
feature_names = ['Weight', 'Engine Size', 'Number of Doors']
importances = rf_classifier.feature_importances_

for name, importance in zip(feature_names, importances):
    print(f"{name}: {importance:.3f}")
```

```
Weight: 0.517
Engine Size: 0.482
Number of Doors: 0.001
```